
Security Review Report

NM-0607 Defx LST



NETHERMIND
SECURITY

(November 24, 2025)

Contents

1	Executive Summary	3
2	Audited Files	4
3	Summary of Issues	4
4	System Overview	5
4.1	Common contracts	5
4.1.1	VaultFactory	5
4.1.2	EmergencyController	6
4.2	Vault Contracts	6
4.2.1	LSTokenVault	6
4.2.2	LSToken	6
4.2.3	UnstakeManager	6
4.2.4	TokenSilo	7
4.2.5	VaultManager	7
4.3	Pricing mechanism	7
5	Risk Rating Methodology	8
6	Issues	9
6.1	[Critical] CreateVault will revert due to lack of permission	9
6.2	[Critical] Incorrect price for depositors during vesting	10
6.3	[Critical] Vault creation will revert with custodians	11
6.4	[Critical] Withdrawals are blocked due to incorrect role assignment	11
6.5	[High] Default limits are referring to 18 decimals tokens	12
6.6	[High] Fees can be lost	12
6.7	[High] Funds are stuck if user claims during vesting period	13
6.8	[High] Funds could be stuck in UnstakeManager during deposit pause	14
6.9	[High] Important admin functions can't be called	15
6.10	[High] Incorrect amount of minted LST during request cancellation	16
6.11	[High] Reverts due to missing ADMIN_ROLE on UnstakeManager	16
6.12	[Medium] Cooldown period changes affect existing requests	17
6.13	[Medium] Daily withdrawal limit is not always applied	17
6.14	[Medium] Early withdraw blocks future withdrawal request	17
6.15	[Medium] Failure to grant DEFAULT_ADMIN_ROLE can permanently lock contract functionality	18
6.16	[Medium] Incompatibility with certain ERC20 Tokens Due to approveUnstakeManager	18
6.17	[Medium] Lack of protection against zero values in addYield()	18
6.18	[Medium] Missing Pause/Unpause Functionality in LSTokenVault	19
6.19	[Medium] Off-chain signatures will revert after version upgrade	19
6.20	[Medium] Stale configuration data returned by the factory	20
6.21	[Medium] Tracking custodians by custodianId could be inaccurate	20
6.22	[Medium] _cleanupOldRequests can skip elements due to incorrect loop logic	21
6.23	[Medium] depositFor checks for the wrong emergency controller status	21
6.24	[Low] The VaultFactory contract can't manage newly created Vault contracts	22
6.25	[Low] Configuration changes can occur on vault during emergency	22
6.26	[Low] Duplicate custodian entries can be added	22
6.27	[Low] Fee-on-transfer and rebase tokens	23
6.28	[Low] Inefficient _removeFromQueue() function	23
6.29	[Low] Potential to bypass early withdrawal fee	23
6.30	[Low] The maxUserDeposit limit can be easily avoided	23
6.31	[Low] The function _cleanupOldRequests() doesn't work correctly	24
6.32	[Low] VaultManager is not granted the required roles on the contracts it manages	24
6.33	[Low] claimsPaused flag not set on revert	25
6.34	[Info] Prices are expressed in different decimals between vaults	25
6.35	[Info] Storage for LSTokenVault	25
6.36	[Info] Unused _maxTransactionPercentage	26
6.37	[Info] Vault status in the VaultFactory is ineffective	26
6.38	[Info] Vaults can be left without custodians	26
6.39	[Info] setFlashLoanProtection is missing in TokenSilo	27
6.40	[Info] transferCollateral function calls a non-existent function	27
6.41	[Info] conflicting state control between ADMIN and EMERGENCY Roles	28
6.42	[Info] versioning and upgrading are not synchronized	29
6.43	[Best Practice] CEI pattern	29
6.44	[Best Practice] Percentages calculations	29
6.45	[Best Practice] Using gaps	30
6.46	[Best Practices] Lack of Segregation of Duties for Critical Roles	30
6.47	[Best Practices] Missing Function to Update version	30
7	Documentation Evaluation	31
8	Test Suite Evaluation	32

8.1	Tests Output	32
8.2	Automated Tools	34
8.2.1	AuditAgent	34
9	About Nethermind	36

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for [Defx LST](#) protocol contracts.

LST is a liquid staking protocol for users to deposit assets to earn yield. For the deposited funds, the user gets a yield-bearing liquid staking token, **LSToken** in return. For every underlying asset, the protocol has a separate vault. New assets can be supported by the deployment of new vaults. Each vault has its own unique **LSToken**.

The vault earns yield by transferring the received funds to other protocols, referred to as custodians. The vault is configured to allocate funds to multiple yield-generating protocols and retain a portion of the funds within the vault to maintain liquidity to meet withdrawal requirements. Periodically, the protocol manager will aggregate the yield generated from all these protocols offline and add the yield amount to the vault. This yield is distributed through a time-vested index mechanism, resulting in a gradual increase in the price of the **LSToken** over time. This is how users earn from their deposits.

The protocol implements a delay in the withdrawal process, primarily due to the time required to unwind the deposited funds from external protocols. When a user submits a request to withdraw their funds, the **LSToken** is burnt and the withdrawal request is added to a queue. The requests in the queue are manually processed by the protocol. Users can claim their funds after the cooling period has passed from the time the request was initiated. Users can choose to bypass the cooling period by paying an early withdrawal fee.

In order to address any unexpected scenarios, the protocol implements a centralized Emergency Controller that can execute a pause on the deposits, withdrawal, or a full pause on the functionalities of the protocol.

During the last part of the review, the client identified additional issues, which were fixed after consultation.

The final repository does not contain the history of the changes since the code without history was added in [defx-contract](#). The last commit of the security review and the history of changes were done in a private repository [lst-contract](#).

The audit comprises 2032 lines of the Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 47 points of attention. where four are classified as Critical, seven are classified as High, twelve are classified as Medium, ten are classified as low and fourteen are classified as Informational or Best Practices severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

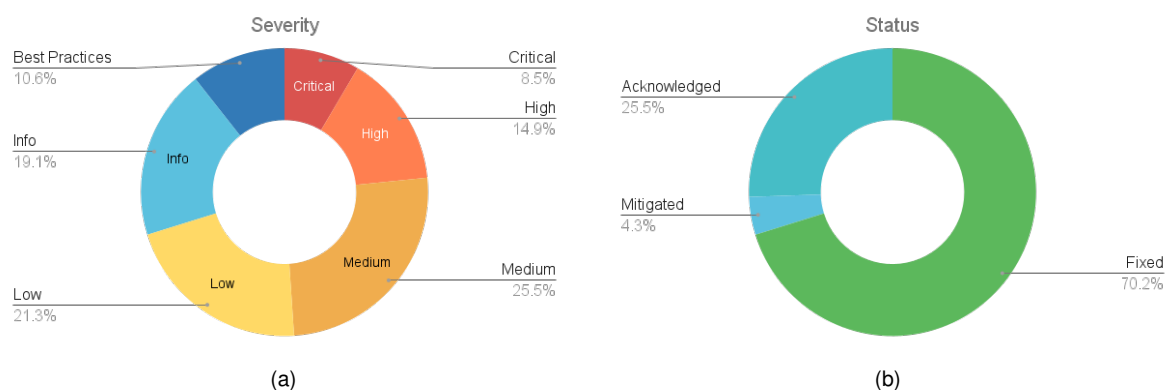


Fig. 1: Distribution of issues: Critical (4), High (7), Medium (12), Low (10), Undetermined (0), Informational (9), Best Practices (5). Distribution of status: Fixed (33), Acknowledged (12), Mitigated (2), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	September 12, 2025
Final Report	November 24, 2025
Initial Commit	3c0e6b474b5adb6496adce68b22a1d45bddb948b
Final Commit	9717185d5822a9f4aa0b39d6f5d7ee4a90f5d36b
Final Commit after merge	a652cf212555c7207bd78af423979047457e1ced
Documentation Assessment	High
Test Suite Assessment	Medium

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	LSToken.sol	80	75	93.8%	23	178
2	VaultManager.sol	142	26	18.3%	31	199
3	UnstakeManager.sol	455	121	26.6%	90	666
4	EmergencyController.sol	119	99	83.2%	37	255
5	LSTokenVault.sol	296	130	43.9%	76	502
6	LSTokenVaultStorage.sol	228	76	33.3%	58	362
7	TokenSilo.sol	377	85	22.5%	70	532
8	VaultFactory.sol	335	29	8.7%	73	437
	Total	2032	641	31.5%	458	3131

3 Summary of Issues

	Finding	Severity	Update
1	CreateVault will revert due to lack of permission	Critical	Fixed
2	Incorrect price for depositors during vesting	Critical	Fixed
3	Vault creation will revert with custodians	Critical	Fixed
4	Withdrawals are blocked due to incorrect role assignment	Critical	Fixed
5	Default limits are referring to 18 decimals tokens	High	Fixed
6	Fees can be lost	High	Fixed
7	Funds are stuck if user claims during vesting period	High	Fixed
8	Funds could be stuck in UnstakeManager during deposit pause	High	Fixed
9	Important admin functions can't be called	High	Fixed
10	Incorrect amount of minted LST during request cancellation	High	Fixed
11	Reverts due to missing ADMIN_ROLE on UnstakeManager	High	Fixed
12	Cooldown period changes affect existing requests	Medium	Acknowledged
13	Daily withdrawal limit is not always applied	Medium	Acknowledged
14	Early withdraw blocks future withdrawal request	Medium	Fixed
15	Failure to grant DEFAULT_ADMIN_ROLE can permanently lock contract functionality	Medium	Mitigated
16	Incompatibility with certain ERC20 Tokens Due to approveUnstakeManager	Medium	Fixed
17	Lack of protection against zero values in addYield()	Medium	Fixed
18	Missing Pause/Unpause Functionality in LSTokenVault	Medium	Fixed
19	Off-chain signatures will revert after version upgrade	Medium	Fixed
20	State configuration data returned by the factory	Medium	Fixed
21	Tracking custodians by custodianId could be inaccurate	Medium	Acknowledged
22	_cleanupOldRequests can skip elements due to incorrect loop logic	Medium	Fixed
23	depositFor checks for the wrong emergency controller status	Medium	Fixed
24	The VaultFactory contract can't manage newly created Vault contracts	Low	Fixed
25	Configuration changes can occur on vault during emergency	Low	Fixed
26	Duplicate custodian entries can be added	Low	Fixed
27	Fee-on-transfer and rebase tokens	Low	Acknowledged
28	Inefficient _removeFromQueue() function	Low	Acknowledged
29	Potential to bypass early withdrawal fee	Low	Fixed
30	The maxUserDeposit limit can be easily avoided	Low	Acknowledged
31	The function _cleanupOldRequests() doesn't work correctly	Low	Fixed
32	VaultManager is not granted the required roles on the contracts it manages	Low	Acknowledged
33	claimsPaused flag not set on revert	Low	Fixed
34	Prices are expressed in different decimals between vaults	Info	Mitigated
35	Storage for LSTokenVault	Info	Acknowledged
36	Unused _maxTransactionPercentage	Info	Fixed
37	Vault status in the VaultFactory is ineffective	Info	Fixed
38	Vaults can be left without custodians	Info	Acknowledged
39	setFlashLoanProtection is missing in TokenSilo	Info	Fixed
40	transferCollateral function calls a non-existent function	Info	Fixed
41	conflicting state control between ADMIN and EMERGENCY Roles	Info	Fixed
42	versioning and upgrading are not synchronized	Info	Fixed
43	CEI pattern	Best Practices	Acknowledged
44	Percentages calculations	Best Practices	Acknowledged
45	Using gaps	Best Practices	Fixed
46	Lack of Segregation of Duties for Critical Roles	Best Practices	Acknowledged
47	Missing Function to Update version	Best Practices	Fixed

4 System Overview

The LST protocol is a decentralized liquid staking system that allows users to stake underlying assets (like rETH) in exchange for yield-bearing Liquid Staking Tokens (LST). The system is designed as a modular and upgradeable architecture that separates concerns across multiple smart contracts. Fig. 1 shows an overview of Defx LST.

The VaultFactory contract is responsible for creating new vault instances for different underlying assets. As part of creating new instances, the VaultFactory sets up roles and permissions for vault smart contracts. It also configures the allocation of funds between custodians and portions to contribute to the liquidity in the vault.

The EmergencyController is a system-wide emergency controller, functions like a global kill switch, offering a multi-tiered emergency response.

Each vault instance comprises its own instances of LSTVault, LSToken, UnstakeManager, and TokenSilo contracts.

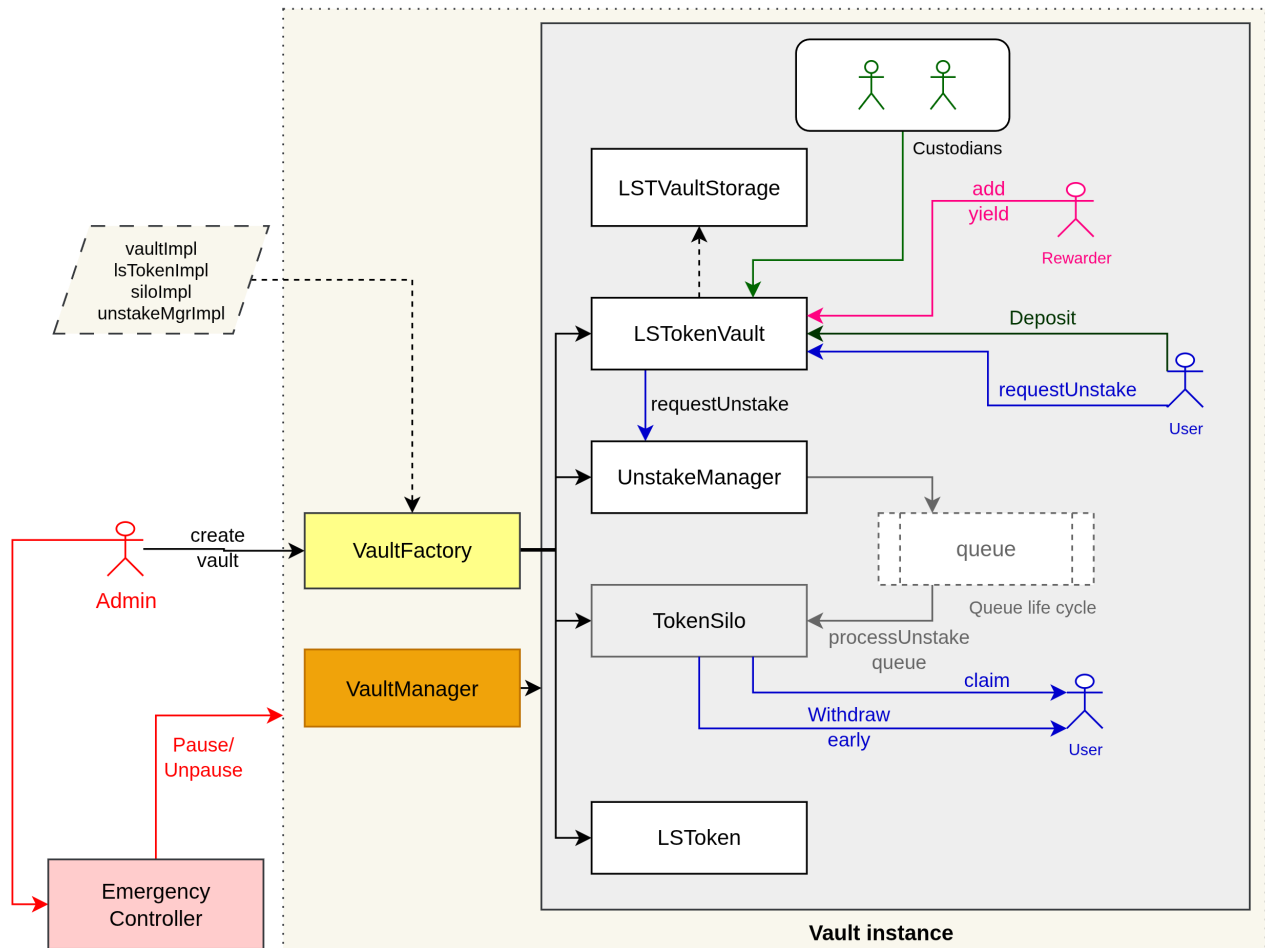


Fig. 2: Defx LST overview

4.1 Common contracts

The following smart contracts are common across the deployed vault instances.

4.1.1 VaultFactory

The factory contract deploys and configures complete vault systems for each underlying asset.

a) **Key functions:**

- creates new vault instances for different underlying assets;
- sets up roles and permissions;
- configures custodian allocations;

b) **Entry points:**

- `createVault()` to deploy a new vault system.

4.1.2 EmergencyController

A global safety mechanism with the authority to pause deposits, withdrawals, or the entire protocol. It provides a centralized kill switch to protect the system during critical events.

a) **Key functions:**

- multi-tier emergency response (deposits paused, withdrawals paused, full pause);
- recovery mode with 24-hour timelock;
- circuit breaker functionality.

b) **Entry points:**

- `pauseDeposits()` / `pauseWithdrawals()` / `pauseAll()` for different modes of pausing the functionality;
- `triggerCircuitBreaker()` for immediate full pause and recovery mode scheduling;
- `activateRecoveryMode()` for highly restrictive mode after timelock.

4.2 Vault Contracts

Every time a new vault instance is deployed by the `VaultFactory`, instances of the following contracts are also deployed to render the functionality of the vault.

4.2.1 LSTokenVault

The central contract for each staked asset. It handles user deposits, mints `LSToken`, manages the distribution of yield via an increasing index, and transfers a portion of deposited funds to off-chain custodian wallets for management.

a) **Key functions:**

- manages core accounting and yield distribution;
- handles custodian fund transfers;
- implements a time-vested index mechanism (8-hour linear vesting);
- manages rate limiting for deposits and withdrawals.

b) **Entry points:**

- `deposit()` for users to stake underlying tokens;
- `requestUnstake()` for users to initiate the withdrawal process;
- `addYield()` for the manager to distribute staking rewards.

4.2.2 LSToken

ERC20 token representing the user's share in the vault.

a) **Key functions:**

- value appreciation as yield is added to the system;
- EIP-2612 permits support for gasless approvals;
- role-based minting/burning.

b) **Entry points:**

- `mint()` is called by vault during deposits
- `burnFrom()` is called by unstake manager during withdrawals

4.2.3 UnstakeManager

This contract orchestrates the unstaking process. When a user wishes to withdraw, their `LSTokens` are burned, and a request is added to a queue. An off-chain manager processes this queue manually.

a) **Key functions:**

- queues unstake requests;
- processes requests (moves funds from vault to silo);

- handles the cooldown period management.

b) **Entry points:**

- requestUnstake() is called by vault
- processUnstakeQueue() is called by managers process requests;
- claim() is called by users to claim funds after the cooldown period.

4.2.4 TokenSilo

A temporary holding contract that secures funds during the unstaking cooldown period. Once an unstake request is processed, the corresponding underlying assets are moved here. Users can claim their funds from the silo after the cooldown expires or opt for an early withdrawal for a fee.

a) **Key functions:**

- segregates withdrawal funds from main vault liquidity;
- supports early withdrawal with fees;
- implements liquidity threshold protection.

b) **Entry points:**

- depositFor() to receive funds from unstake manager/
- withdrawTo() to transfer funds to users/
- earlyWithdraw() for users to withdraw early by paying a fee.

4.2.5 VaultManager

This is a configuration manager contract for the vault. This is a single point from where configuration can be managed for all the underlying smart contracts of the vault.

a) **Key functions:**

- proxies admin functions to vault;
- manages vault parameters.

4.3 Pricing mechanism

Defx LST application developed a custom pricing and yield distribution mechanism. The yield accrued by the trusted custodian accounts is registered with the addYield() function, restricted to a special role. The amount of yield is divided by the total supply of the LST tokens, and the targetIndex is calculated. However, this target price is not currently available, but is vested. It is performed by tracking two values: initialIndex and targetIndex. Fig. 3 shows the Defx LST price mechanism. During the pre-defined window of time, the price increases proportionally from the initialIndex and targetIndex. If the user would like to withdraw from the protocol, the price of their LST would be calculated by the current price, which depends on how much time has passed from the last yield addition, and is in the range [initialIndex, targetIndex]. Once the targetIndex is achieved, a new yield can be registered.

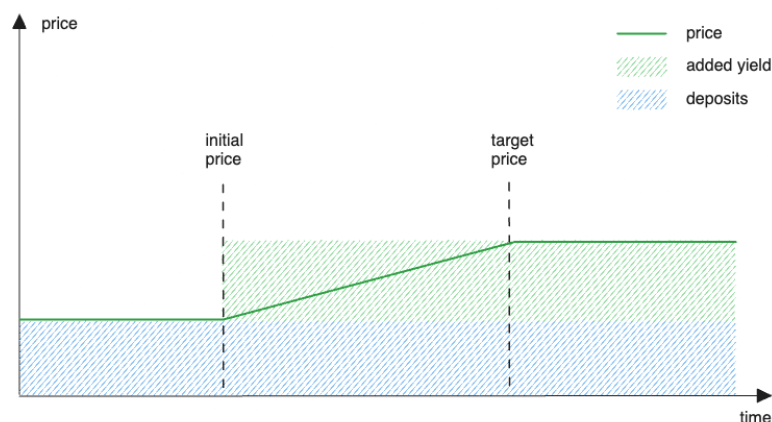


Fig. 3: Defx LST price mechanism

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Critical] CreateVault will revert due to lack of permission

File(s): LSTokenVault.sol, VaultFactory.sol

Description: When creating a new Vault in VaultFactory by calling createVault function, As the VaultFactory was not configured with ADMIN_ROLE, the below calls to configure the UnstakeManager and EmergencyController will revert.

```
1  function createVault(  
2      address underlyingToken,  
3      string memory underlyingSymbol,  
4      string memory lsTokenName,  
5      string memory lsTokenSymbol,  
6      TokenConfig memory config  
7  ) external onlyRole(ADMIN_ROLE) returns (  
8      //...  
9  ) {  
10     //...  
11     // Setup roles  
12     _setupRoles(vaultAddress, lsTokenAddress, siloAddress, unstakeManagerAddress);  
13  
14     // Configure contracts  
15 ==> ILSTokenVault(vaultAddress).setUnstakeManager(unstakeManagerAddress);  
16 ==> ILSTokenVault(vaultAddress).setEmergencyController(address(globalEmergencyController));  
17     ITokenSilo(siloAddress).setEmergencyController(address(globalEmergencyController));  
18     //...  
19 }
```

This will block the deployment of a new vault.

A similar problem exists in the _applyConfig function, which calls MANAGER_ROLE functions on the vault.

Recommendation(s): Consider granting the Admin role to VaultFactory in the initialize function. Also, consider reviewing the roles and permissions configured in the protocol.

Status: Fixed

Update from the client: This finding is no longer applicable as the VaultFactory.sol contract has been deprecated and removed from our protocol architecture, resolving this issue. Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.2 [Critical] Incorrect price for depositors during vesting

File(s): LSTokenVault.sol

Description: The standard ERC4626 vault works in a way that increases the value of the LST at the same moment when the yield is supplied to the vault. Newcomers who deposit always buy a token at the price which includes the yield. The DEFx vault distributes the yield among LST holders using a vesting mechanism, so that the yield defines the target price. Over time, the price increases from the initial price to the target price, eventually reaching the full target price, unlocking the whole yield. This mechanism incentivises users to hold LST, at least until the vesting ends. However, the current mechanism does not correctly account for newly deposited users. New depositors buy LST tokens at the current price, i.e. somewhere between the initial price and target price. This means that new depositors could enter the staking in the middle of the vesting and benefit from the yield generated by previous stakers. This should not happen, since the new depositors are stealing the old depositors' yield. Moreover, substantial amounts from new depositors may block old depositors from withdrawing, since the target price would point to a greater amount of underlying tokens than there actually is. Consider the following example:

- $\text{lst_amount} = 100$, $\text{underlying_amount} = 100$, $\text{initial_index} = 1$;
- yield added 100, so $\text{underlying_amount} = 200$, $\text{target_index} = 2$, since yield added (100) divided by lst_amount (100) is 1, which is added to initial_index (1) resulting in $\text{target_index} = 2$;
- only one user A is holding 100 LST;
- after some time, the $\text{current_index} = 1.6$;
- new user B deposits 1_000 tokens at $\text{current_index} = 1.6$, resulting in $1_000 / 1.6 = 625$ LST tokens;
- the vesting period ends and current_index equals $\text{target_index} = 2$;
- user's A LST tokens are worth $100 * 2 = 200$ of underlying;
- user's B LST tokens are worth $625 * 2 = 1250$;
- total worth of LST tokens based on above calculation is $1250 + 200 = 1450$;
- the actual amount of underlying tokens is user A deposit + user B deposit + yield, which is $100 + 1_000 + 100 = 1200$;
- the 250 underlying tokens are missing, if user B withdraws the whole available amount, user A won't be able to withdraw;

Recommendation(s): Consider allowing deposits only at the target price. This would allow the avoidance of incorrect calculations described above. Note, however, that the new depositors could profit only after new yield additions. If the new depositor who deposited mid-vesting would like to withdraw before the target price, they would lose money, since the deposit happened at the target price and withdrawal at the current price. If the recommended fix is applied, to prevent losses in this scenario, consider either clearly informing users or implementing a withdrawal lock period greater than the vesting period.

Alternative solutions could be to vest yield instead of vesting the price. This would allow for the calculation of the correct current price for both withdrawing and depositing based on the current amount of LST supply. Consider the following example in this scenario:

- $\text{lst_amount} = 100$, $\text{underlying_amount} = 100$, $\text{initial_price} = 1$;
- yield added 100, yield will be released over the vesting period;
- only one user A is holding 100 LST;
- after some time, the released yield is 60%, so the total underlying is 160;
- new user B deposits 1_000 tokens, current price is $p = 160 \text{ (underlying)} / 100 \text{ (lst)} = 1.6$, resulting in $1_000 / 1.6 = 625$ LST tokens;
- the vesting period ends and whole yield (100) is released, the current price is $p = 1200 \text{ (underlying)} / 725 \text{ (lst)} = 1.655$;
- user's A LST tokens are worth $100 * 1.655 = 165.5$ of underlying;
- user's B LST tokens are worth $625 * 1.655 = 1034.375$;
- total worth of LST tokens based on above calculation is $165.5 + 1034.375 = 1199.875$, which is very close to 1200 (real amount of underlying);

Note that in this scenario, user A earns less, however, the vesting of yield still motivates users to hold tokens until the vesting period ends.

Status: Fixed

Update from the client: To address the incorrect pricing issue for depositors during yield vesting, we've implemented a two-part fix. First, deposits are now calculated using the post-yield `targetIndex`, ensuring fair value for all stakers. Second, we've introduced a hard withdrawal lock equal to the yield vesting period that activates on any new deposit. This prevents users from withdrawing at a loss and secures the protocol's accounting. Commits: [42c7a9eb98a43b9378889b34ce431516b5c7658f](#) and [d865da3381427232ce19b3dad77ecd8a5ce365a7](#)

6.3 [Critical] Vault creation will revert with custodians

File(s): VaultFactory.sol, LSTVault.sol, LSTVaultStorage.sol

Description: The createVault function is used by the ADMIN_ROLE to deploy a new vault and its associated contracts, including the LSTokenVault, TokenSilo, and UnstakeManager. The function takes a TokenConfig struct, which contains information about custodians and their allocations. The _setupCustodians function is called to configure these custodians on the newly deployed LSTokenVault contract.

```

1  function createVault(
2      //...
3      TokenConfig memory config
4  ) external onlyRole(ADMIN_ROLE) returns (
5      //...
6  ) {
7      //...
8      // Validate custodian configuration
9      require(config.custodians.length <= 10, "Too many custodians");
10     require(config.custodians.length > 0, "Must have at least one custodian");
11     //...

```

The createVault function expects to have at least one custodian passed in the configuration parameter.

The _setupCustodians function is intended to add all the custodians provided in the config struct to the vault. It iterates through the config.custodians array. For the first custodian (at index 0), it calls the vaultContract.updateCustodian function. For all subsequent custodians, it calls the vaultContract.addCustodian function.

The updateCustodian function is designed to modify an existing custodian at a given index. However, when it is called for the very first custodian, the LSTokenVault contract's custodians array is empty. The _updateCustodian function, which is called by the updateCustodian external function, contains a required statement at the beginning that checks if the provided custodianId is a valid index in the custodians array. Since custodians.length is 0 at this point, the check custodianId < custodians.length (0 < 0) will fail, causing the transaction to revert.

```

1  function _updateCustodian(uint256 custodianId, address wallet, uint256 allocationPercent) internal {
2  ==>   require(custodianId < custodians.length, "Invalid custodian ID");
3      //...
4  }

```

This vulnerability means that the _setupCustodians function will always fail and revert the entire createVault transaction, making it impossible to create any new vaults. This denial of service attack affects a core functionality of the protocol.

Recommendation(s): Consider refactoring the _setupCustodians function to use addCustodian for all custodians, as this function is designed to append new custodians to the array.

Status: Fixed

Update from the client: This finding is now resolved as the VaultFactory.sol contract has been deprecated and removed from the protocol architecture, eliminating the faulty createVault function entirely. Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.4 [Critical] Withdrawals are blocked due to incorrect role assignment

File(s): VaultFactory.sol, UnstakeManager.sol, TokenSilo.sol

Description: During VaultFactory.createVault() the TokenSilo is deployed and the VaultFactory assigns the roles. The VAULT_ROLE in TokenSilo, which is responsible for calling depositFor() and withdrawTo() during the unstake process, is given to LSTokenVault. However, functions depositFor() and withdrawTo() are called from the UnstakeManager contract. As a result, when UnstakeManager tries to perform those calls during the unstaking process, the calls would fail and the unstaking process wouldn't be possible. A similar issue is with the MANAGER_ROLE in UnstakeManager, which is assigned to the VaultFactory. The functions that require MANAGER_ROLE in UnstakeManager are responsible for processing unstake requests: markRequestsForProcessing(), processUnstakeQueue(), processUserUnstake(), cancelUnstake(). Those functions are not called from VaultFactory, therefore, withdrawals are blocked.

Recommendation(s): Consider correctly assigning the VAULT_ROLE in TokenSilo to UnstakeManager, and the MANAGER_ROLE in UnstakeManager to address those connected with off-chain operators who process the unstake queue. Consider also changing the name of the role VAULT_ROLE in TokenSilo so that it better expresses the caller.

Status: Fixed

Update from the client: Acknowledged. This finding is no longer applicable. The VaultFactory.sol contract, which was responsible for these incorrect role assignments, has been deprecated and removed from our protocol architecture, resolving this issue.

Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.5 [High] Default limits are referring to 18 decimals tokens

File(s): LSTokenVaultStorage.sol, LSTokenVault.sol

Description: The function `_setupDefaults()` is called during LSTVault initialization to set up initial values. However, those values refer to the tokens with 18 decimals. If the underlying tokens have fewer decimals, e.g. 6, then those limits are much larger than expected. This affects minimum amount checks during deposit and withdrawal, which are initialised with: `MIN_DEPOSIT_AMOUNT = 0.1 ether`, `minUnstakeAmount = 0.1 ether`. If the underlying token has fewer decimals than 18, e.g. 6 in USDC, the minimum amount to deposit would be `1e11` tokens, which effectively makes the vault unusable. Other affected variables that are expressed in 18 decimals: `maxTotalDeposit`, `maxUserDeposit`, `depositLimit`. This affects total deposits limits, and the `_validateRateLimit()` function, where it is assumed that deposit limit is twice greater than withdrawal limit (`100_000 * 10e18`, `50_000 * 10e18`). But if the underlying token has 6 decimals, the deposit limit would be `2 * 10e12` times greater than the withdrawal. Note that a similar situation is in TokenSilo for values like `maxDailyAmount`.

Recommendation(s): Consider setting limiting values without the decimals, and apply decimals of the underlying token when needed. Additionally, consider storing those settings in a way that allows modification by the admin.

Status: Fixed

Update from the client: Updated contract initialisation to dynamically scale default limits based on the underlying token's decimals. Commit : [d8b0ce0a2a09e7263a4b13d88f4c81cab13548b3](#) and [9717185d5822a9f4aa0b39d6f5d7ee4a90f5d36b](#)

6.6 [High] Fees can be lost

File(s): UnstakeManager.sol, LSTokenVault.sol

Description: The fees deducted during early withdrawal in TokenSilo are sent to the `feeCollector` which is set to `VaultFactory` during initialization. Since `VaultFactory` does not have a mechanism to transfer tokens, those fees are lost. Moreover, during yield addition in LSTokenVault the fee is not registered if the `feeReceiver` is not set, but the fee amount is still deducted from the yield amount, resulting in losing the fee:

```

1  function addYield(uint256 yieldAmount) external onlyRole(REWARDER_ROLE) {
2      ...
3      // @audit: fee calculated
4      uint256 feeAmount = calculatePercentage(yieldAmount, feePercent, PERCENT_PRECISION);
5      // @audit: underlying amount is decreased by the fee
6      uint256 distributableYield = yieldAmount - feeAmount;
7      // @audit: fee is not registered if feeReceiver is not set
8      if (feeAmount > 0 && feeReceiver != address(0)) {
9          totalFeeCollected += feeAmount;
10         emit FeesCollected(feeAmount);
11     }
12
13     totalCustodianFunds += yieldAmount;
14
15     uint256 current = getCurrentIndex();
16     // @audit: new price calculated based on decreased yield
17     uint256 deltaIndex = calculatePercentage(distributableYield, INDEX_PRECISION, supply);
18     ...
19 }
```

Note that the `feeReceiver` in LSTokenVault is not set during the vault creation it must be called separately with `setFeeReceiver()` function, which can be called by the manager role.

Recommendation(s): Consider ensuring the fees are not lost by setting the correct fee receiver during initialisation.

Status: Fixed

Update from the client: 1. TokenSilo initialisation now requires a specific `feeCollector` address, preventing fees from being sent to an inaccessible contract. 2. LSTokenVault's `addYield` logic is updated to ensure yield fees are only deducted when they are also collected. Commit : [208902c9d4a09485aa7be1a66c00b7d3fdea863c](#)

6.7 [High] Funds are stuck if user claims during vesting period

File(s): LSTokenVault.sol

Description: The LSTokenVault calculates the new index (price) when the yield is supplied. This yield however, is vested and released proportionally to the defined time window. If the user claims before the whole yield is released, those funds are lost, since the unclaimed yield is not redistributed among other LST token holders. Consider the following scenario (note that the scenario is simplistic, e.g. zero fees):

- `lst_amount` = 100, `underlying_amount` = 100, `current_index` = 1;
- yield added 100, so `underlying_amount` = 200, `target_index` = 2;
- two users hold LST tokens, each 50;
- user A burns 50 LST after some time the addition of the yield at `current_index` = 1.5, equating to $50 * 1.5 = 75$ underlying tokens;
- user B waits whole vesting period and burns 50 LST at `current_index` = 2, equating to $50 * 2 = 100$ underlying tokens;
- the sum of withdrawn amount is 175, while 25 underlying tokens stays in the Vault;

Recommendation(s): Consider redistributing the amount that is left when withdrawing during the vesting period. This could be done by recording the difference between `unclaimed` = $(\text{user_lst_amount} * \text{target_index}) - (\text{user_lst_amount} * \text{current_index})$ and adding this unclaimed amount to the `distributableYield` during yield addition. This would allow for redistributing unclaimed funds, while preserving the incentive to hold LST with vesting and protecting from losing the unclaimed funds. Alternatively, the unclaimed funds could be used as additional protocol revenue, which would require additional changes and implementing mechanisms to allow the owner to withdraw those funds. Another solution could be to vest the yield amount instead of vesting the price, as described in the previous issue.

Status: Fixed

Update from the client: To resolve the issue of forfeited yield becoming stuck in the vault, we have implemented a redistribution mechanism. When a user unstakes before a vesting period is complete, the value they forfeit is now captured and pooled. This pooled amount is then automatically added to the next `addYield` event, ensuring the value is fairly redistributed to all remaining stakers and eliminating the value leak. Commits: [0c94bfa9d44e5ae2c4cc4519db03bdf32b40b37b](#) and [594ab3dd22ade90813d8c9d0a4c3cdf973c89b4](#)

6.8 [High] Funds could be stuck in UnstakeManager during deposit pause

File(s): UnstakeManager.sol

Description: The UnstakeManager contract is responsible for processing unstake requests from a queue in batches. The processUnstakeQueue function is called to handle these requests. It first calculates the total amount to be processed for a batch and pulls the funds from a Vault contract.

```

1  for (uint256 i = 0; i < queuedRequestIds.length && batchIndex < batchSize; i++) {
2      //...
3      if (totalAmountToProcess + request.underlyingAmount <= vaultBalance) {
4          //...
5          totalAmountToProcess += request.underlyingAmount;
6          batchIndex++;
7      }
8  }
9
10 if (batchIndex > 0 && totalAmountToProcess > 0) {
11     ==> _transferFromVault(totalAmountToProcess);

```

The function then iterates through the queued unstake requests and attempts to deposit the funds into the appropriate TokenSilo contract using the depositFor function. This is done within a try/catch block.

```

1  // Iterate through the queued requests and process them.
2  for (uint256 i = 0; i < batch.length; i++) {
3      UnstakeQueueRequest memory request = batch[i];
4      // @audit Unstake requests are processed in a try/catch block
5      try ITokenSilo(request.tokenSilo).depositFor(
6          request.account, request.amount
7      ) {
8          // ...
9          unstakeRequests[requestHash].status = UnstakeStatus.PROCESSED;
10     } catch {
11         // @audit The transaction will continue even if `depositFor(...)` reverts.
12         emit UnstakeQueueRequestFailed(request);
13     }
14 }
15 // ...

```

if the EmergencyController has paused deposits (by setting DEPOSITS_PAUSED to true), the depositFor function will revert. Because the call is wrapped in a try/catch block, the processUnstakeQueue transaction will not revert. Instead, it will emit a UnstakeQueueRequestFailed event for each failed deposit and continue to the next request.

The critical issue is that while the funds are successfully pulled from the Vault at the beginning of the function, they are not deposited into the TokenSilo contracts due to the deposit pause. The status of each request is not marked as PROCESSED, so they will remain in the queue. The funds, however, are now stuck in the UnstakeManager contract, making them unavailable to process future unstake requests. The UnstakeManager contract has no mechanism to recover or return these stuck funds.

Recommendation(s): Unify the conditions for withdrawals between UnstakeManager and TokenSilo contracts. In case the depositFor reverts, in the catch block, the unstake request funds should be returned to the Vault contract.

Also, after exiting the loop, set the approved amount to 0 to ensure that unprocessed approval is reset to 0.

Status: Fixed

Update from the client: The processUnstakeQueue and processUserUnstake functions are updated to:

1. Unify emergency checks to verify DEPOSITS_PAUSED state before execution;
2. Wrap the silo.depositFor call in a try/catch block;
3. In the catch block, safely return funds to the LSTokenVault if the deposit fails, ensuring no assets are trapped;
4. Reset token approvals to zero after execution in both success and failure paths to enhance security;
5. Ensure all state changes (e.g., queue accounting) only occur on a successful deposit;

Commit: [7ff0110a062724fe377d806a467dd0d42680a833](#)

6.9 [High] Important admin functions can't be called

File(s): [VaultFactory.sol](#), [UnstakeManager.sol](#), [TokenSilo.sol](#), [LSToken.sol](#)

Description: The functions that are responsible for upgrading contracts can't be called in UnstakeManager, TokenSilo, LSToken, since ADMIN_ROLE is assigned to VaultFactory, which can't call those functions. This makes the above contracts unable to upgrade. While some functions are called from VaultFactory, below we list the functions which can't be called from VaultFactory, therefore are unusable: UnstakeManager:

- requestUpgrade();
- cancelUpgrade();
- _authorizeUpgrade();
- updateVersion();

TokenSilo:

- requestUpgrade();
- cancelUpgrade();
- _authorizeUpgrade();
- updateVersion();
- setVault();
- rescueTokens();
- pause();
- unpause();
- resetDailyLimit();
- setRateLimit();
- adjustPendingClaims();
- setFeeCollector();
- setEarlyUnlockEnabled();
- setUnlockFee();
- setLiquidityThreshold();
- setClaimsPaused();

LSToken:

- requestUpgrade();
- cancelUpgrade();
- _authorizeUpgrade();
- updateVersion();

Recommendation(s): Consider correctly assigning ADMIN_ROLE to the address that can perform admin actions.

Status: Fixed

Update from the client: This finding is no longer applicable. The VaultFactory.sol contract, which was responsible for these incorrect role assignments, has been deprecated and removed from our protocol architecture, resolving this issue.

Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.10 [High] Incorrect amount of minted LST during request cancellation

File(s): `LSTokenVault.sol`, `UnstakeManager.sol`

Description: During unstake request creation, users provide an amount of LST, from which the underlying amount is calculated at the current price. Amounts of LST and underlying are stored in the request object, and the LST tokens are burned. The application contains the `cancelUnstake()` function, which allows the manager to cancel the unstake request. Cancellation is performed by removing requests from relevant arrays and mappings, and minting back the LST. However, this causes incorrect calculations in specific conditions. A user creating a mistake request stores information about the amount of LST and the underlying. After this event, the yield could be added and distributed over the whole LST supply, without the LST being burned by the user, which creates a new target price. If the manager were to now cancel the request created before yield, that action would be equivalent to buying the LST at the stale, lower price, from the time of request creation. That would break the yield distribution and could lead to protocol insolvency. A similar problem of buying a token for a cheaper price is described in the issue: "[Critical] Incorrect price for depositors during vesting".

Recommendation(s): During cancellation, instead of minting the frozen amount from the time of unstake request creation, consider calculating a new amount of LST from the request underlying amount based on the target price. Note that the target price should be used even if it is not reached yet, to avoid problems described in "[Critical] Incorrect price for depositors during vesting". Effectively, the cancellation should be treated as depositing a new underlying amount, registered in the request.

Status: Fixed

Update from the client: The `cancelUnstake` function in `UnstakeManager.sol` has been updated to remediate the reported vulnerability. The function no longer mints the stale `IsTokenAmount` stored at the time of the request. Instead, it now treats the cancellation as a new deposit. The logic fetches the current `targetIndex` from the `LSTokenVault` and recalculates the amount of `LSTokens` to mint based on the user's original `underlyingAmount`. This change ensures that the user receives an amount of `LSTokens` that accurately reflects the current asset value, preventing the exploit and protecting the protocol from value drain. The `ILSTokenVault` interface was also updated to expose the `targetIndex` for this purpose. Commit: [cee8bd902ecd97210ec9c5f7afce0fc95b036ad1](#) and [e1afb1f13f271395c971615b50ab89d9afa0c87c](#)

6.11 [High] Reverts due to missing ADMIN_ROLE on UnstakeManager

File(s): `UnstakeManager.sol`

Description: The `UnstakeManager` contract overrides the `grantRole(...)` function from `AccessControlUpgradeable`. This function is designed to allow an admin to grant a specific role to a new account. The override correctly checks that the caller has the necessary admin rights for the role being granted.

```
1 function grantRole(bytes32 role, address account) public override(AccessControlUpgradeable, IUnstakeManager)
2   → onlyRole(getRoleAdmin(role)) {
3     // @audit-issue The UnstakeManager contract itself must have the ADMIN_ROLE to call super.grantRole.
4     super.grantRole(role, account);
5   }
```

The issue is that the overridden function internally calls `super.grantRole(...)`, which executes the `grantRole(...)` logic from the parent `AccessControlUpgradeable` contract. In this context, the caller of `super.grantRole(...)` is not the external wallet that initiated the transaction, but the `UnstakeManager` contract itself. According to OpenZeppelin's `AccessControl` implementation, only an address with the `ADMIN_ROLE` for a given role can grant that role. This means that for the `super.grantRole(...)` call to succeed, the `UnstakeManager` contract itself must have the `ADMIN_ROLE` for every role it intends to manage. If the `UnstakeManager` contract is not granted the `ADMIN_ROLE` for the roles it manages, any attempt to call `grantRole(...)` will revert, effectively preventing all role management from this contract.

Recommendation(s): Consider modifying the `grantRole` to use `_grantRole` which is the internal function.

Status: Fixed

Update from the client: Resolved by using the internal `_grantRole` function. Commit: [7741a1e48490ce5505cd054b274fd738c302cdc8](#)

6.12 [Medium] Cooldown period changes affect existing requests

File(s): [UnstakeManager.sol](#)

Description: The UnstakeManager contract includes a cooldown period for withdrawals, meaning users must wait a certain amount of time after submitting a request before they can claim their funds. The `setCooldownPeriod(...)` function allows an authorised administrator to change this period.

```

1 function setCooldownPeriod(uint256 _period) external onlyRole(ADMIN_ROLE) {
2     require(_period > 0 && _period <= maxCooldownPeriod, "UnstakeManager: invalid period");
3     // @audit-issue The new cooldownPeriod is applied to all requests, including existing ones.
4     cooldownPeriod = _period;
5     emit CooldownPeriodSet(_period);
6 }

```

The issue is that the `cooldownPeriod` is a single state variable. When an administrator updates this variable, the new value immediately affects all pending requests. If the administrator increases the cooldown period, users who have already submitted a request will have to wait longer than they initially expected. This could be confusing for users and lead to a poor user experience. Conversely, if the period is decreased, it could allow users to claim funds sooner, which may not align with the protocol's intended flow or might be a security risk if the change is made improperly. The new cooldown period should only apply to new requests submitted after the change.

Recommendation(s): Consider storing the exact timestamp when a user's cooldown period ends, rather than calculating it based on a global variable at the time of claiming in the unstake request.

Status: Acknowledged

Update from the client: This behaviour is intentional and a critical component of our risk management strategy. The cooldown period is a protocol-wide safety parameter, not a fixed-term agreement with individual users. It is designed to be adjustable by the administration in response to dynamic market conditions, liquidity events, or potential security threats. For this tool to be effective, any changes must apply globally to all pending withdrawals. This ensures the protocol can act decisively to protect the interests of all stakers, rather than being locked into variable, user-specific timelines. We consider this dynamic control a necessary feature for maintaining protocol stability.

6.13 [Medium] Daily withdrawal limit is not always applied

File(s): [TokenSilo.sol](#)

Description: The daily withdrawal limit is checked with `_validateRateLimit()`. However, this function is only called in the `earlyWithdraw()` but not in `withdrawTo()`. Therefore, the limit is not effective.

Recommendation(s): Consider calling `_validateRateLimit()` in the `withdrawTo()`.

Status: Acknowledged

Update from the client: We are accepting this risk as a design trade-off. Standard withdrawals in `withdrawTo()` are only possible after the fund is in Silo, which is our primary security gate. Applying a further rate limit after this point would unfairly penalise users who have completed all required steps. The global `EmergencyController` remains our main tool for system-wide halts.

6.14 [Medium] Early withdraw blocks future withdrawal request

File(s): [UnstakeManager.sol](#), [TokenSilo.sol](#)

Description: User is able to call `TokenSilo.earlyWithdraw(...)` to claim tokens before the cooldown period. However, this function does not have the same effect as the `UnstakeManager.claim(...)`, specifically, the user's data in `unstakeRequests` mapping is not removed. The data in `unstakeRequests` mapping is checked during the withdrawal request and should be empty. Therefore, addresses that use early withdrawal won't be able to make new withdrawal requests. Note that eventually, after 30 days, the `unstakeRequests` could be removed in cleanup, which would allow users to deposit with the same address.

Recommendation(s): Consider unifying effects of state updates between `TokenSilo.earlyWithdraw(...)` and `UnstakeManager.claim(...)`. Ideally, users should interact with the `UnstakeManager` contract for both normal and early withdrawal flows.

Status: Fixed

Update from the client: Centralised all withdrawal logic within the `UnstakeManager`. A new `earlyWithdraw` function has been added to the `UnstakeManager`, which is now the single point of contact for users. This function securely calls the `TokenSilo` on the user's behalf and, critically, performs the same state cleanup as a normal claim, ensuring the user's request is properly closed out. Commit: [ee6ae008f1ca4be8478654a65440168ac549a7ea](#)

6.15 [Medium] Failure to grant DEFAULT_ADMIN_ROLE can permanently lock contract functionality

File(s): *.sol

Description: The protocol's contracts utilise a role-based access control system to manage permissions for critical functions. For the system to be operational, various roles must be assigned to the appropriate addresses post-deployment. The authority to manage these roles is controlled by an admin role.

The protocol uses an access control mechanism where the DEFAULT_ADMIN_ROLE is the admin for all other roles by default, as no custom role hierarchy has been established. If the DEFAULT_ADMIN_ROLE is not assigned to a trusted address during the deployment process, no one will have the permission to grant the necessary operational roles to other accounts.

This oversight would render all role-protected functionality permanently inaccessible. For example, if no account holds the MINTER_ROLE, new tokens cannot be minted, and if no one can grant the MINTER_ROLE, this functionality is lost forever.

Recommendation(s): Consider ensuring that the deployment procedure explicitly grants the DEFAULT_ADMIN_ROLE to a secure and trusted address, such as a multisig wallet or a designated globalAdmin account. This administrative account would then be responsible for the initial configuration of all other operational roles.

Status: Mitigated

Update from the client: This is a critical operational requirement. We will update our deployment scripts and documentation to explicitly ensure that the DEFAULT_ADMIN_ROLE is granted to a secure, pre-determined multisig wallet during the initialisation of all relevant contracts. This will be a mandatory and audited step in our deployment checklist.

6.16 [Medium] Incompatibility with certain ERC20 Tokens Due to approveUnstakeManager

File(s): LSTokenVault.sol

Description: The approveUnstakeManager(...) function is intended to approve a specific amount of the underlyingToken for the unstakeManager to spend. This is a critical step in the protocol's workflow. The function uses safeApprove(...) to set the allowance.

The current implementation of approveUnstakeManager(...) is incompatible with certain ERC20 tokens, such as USDT, that do not conform to the standard approve function behaviour. Specifically, some tokens revert if approval is called to change an existing non-zero allowance to another non-zero value without first resetting it to zero.

The safeApprove function wrapper from OpenZeppelin's SafeERC20 library will revert if the underlying token's approve function reverts. As a result, if the unstakeManager already has a non-zero allowance for an incompatible token, any subsequent call to approveUnstakeManager(...) will revert, effectively breaking the functionality. This means the protocol's admin will be unable to update the allowance for the unstakeManager, which could lead to a denial of service for any operations that depend on this allowance.

```

1  function approveUnstakeManager(uint256 amount) external onlyRole(ADMIN_ROLE) {
2      require(address(unstakeManager) != address(0), "Unstake manager not set");
3      // @audit-issue USDT's approve function will revert if a non-zero allowance is already set.
4      underlyingToken.safeApprove(address(unstakeManager), amount);
5  }

```

Recommendation(s): Consider setting the allowance to 0 using safeApprove(...) before setting it to the desired amount.

Status: Fixed

Update from the client: The function now explicitly resets the allowance to zero before setting the new, desired amount. Commit: [edd81e2ba9042a97ac355d7a5fadad194625bbdb](#)

6.17 [Medium] Lack of protection against zero values in addYield()

File(s): LSTokenVault.sol, TokenSilo.sol

Description: The addYield() function calculates protocol fee and deltaIndex. However, if the provided yield amount is low, the fee and deltaIndex could be rounded down, which would result in a lack of registering yield. Note that in deposit and withdrawal requests, the minimum amount of provided tokens is specified, but in addYield() it is not. Such a check is not present in earlyWithdraw() as well.

Recommendation(s): Consider checking if the calculated amounts are not zero and defining the minimum amount of added yield.

Status: Fixed

Update from the client: We have implemented the recommended check. The addYield function now includes a require(deltaIndex > 0) statement to ensure the transaction reverts if the yield amount is too low to register, preventing any loss of funds. Commit: [68f6d0d9528f846bc0c4a1f71ea1af189d5b278e](#)

6.18 [Medium] Missing Pause/Unpause Functionality in LSTTokenVault

File(s): [LstTokenVault.sol](#)

Description: The LSTTokenVault contract inherits from PausableUpgradeable, which provides the infrastructure for pausing and unpausing a contract's operations. This is a crucial security feature that allows a designated role to halt specific functions in case of an emergency, such as a vulnerability or a market anomaly.

The LSTTokenVault contract correctly applies the whenNotPaused modifier to restrict certain functions, ensuring they cannot be executed when the contract is in a paused state. However, the contract lacks the necessary public or external functions to trigger the pause and unpause actions. The PausableUpgradeable contract provides internal functions `_pause()` and `_unpause()`, which must be wrapped in an externally callable, access-controlled function within the LSTTokenVault contract to be useful.

Currently, the only way to pause the vault is through the protocol-wide EmergencyController contract. While this is a valid mechanism for a global emergency, it does not provide the flexibility to pause only a single LSTTokenVault in a multi-vault system. For example, if a specific vault encounters an issue that does not affect the rest of the protocol, the only option is to shut down the entire system, which is an overly broad and disruptive measure.

Recommendation(s): Consider adding access-controlled functions to the LSTTokenVault contract that call the `_pause()` and `_unpause()` functions. These functions should be restricted to a specific role to prevent unauthorised use.

Status: Fixed

Update from the client: Implemented two new external functions, `pause()` and `unpause()`, which are restricted to an `EMERGENCY_ROLE`. Commit: [633e7d8a559d34ea4a33994523aa2b93cf19b125](#)

6.19 [Medium] Off-chain signatures will revert after version upgrade

File(s): [LSToken.sol](#)

Description: The LSToken contract, which extends ERC20PermitUpgradeable, supports off-chain signatures for gasless approvals. This is achieved through the EIP-712 standard, which includes a version string in the domain hash to ensure signatures are valid only for a specific contract version.

The LSToken contract maintains its own version state variable to track its implementation version. However, the EIP-712 implementation from the ERC20PermitUpgradeable library uses its own, internally hardcoded version of "1", which is independent of the contract's state variable. This means that when the contract is upgraded and its own version state variable is incremented to "2", the version used for off-chain signature verification remains unchanged. As a result, the signature generated using the contract version will be invalid.

To prevent this issue, the `_EIP712Version()` function, which is a virtual function in the underlying EIP-712 implementation, must be overridden to return the contract's own version state variable.

```
1 function _EIP712Version() internal virtual view returns (string memory) {  
2     return version;  
3 }
```

Recommendation(s): Consider overriding the `_EIP712Version()` function to use the contract's own version state variable.

Status: Fixed

Update from the client: Implemented the recommended fix by overriding the `_EIP712Version()` function in our LSToken contract to return our own stateful version string. Commit: [e696f5933dae30f60dd9cf1db57ae5da29e6df13](#)

6.20 [Medium] Stale configuration data returned by the factory

File(s): VaultFactory.sol

Description: The VaultFactory contract caches the configuration of each vault in the vaultConfigs mapping. This cached data is intended to be retrieved via the getTokenConfig function.

The problem is that vault parameters can be modified by administrators through other contracts, such as VaultManager or directly by an admin on the target contract. For example, when a setting is changed, for instance by calling setMinUnstakeAmount, the update is applied to the appropriate contract (e.g., UnstakeManager), but the vaultConfigs mapping in VaultFactory is not updated.

The getTokenConfig function contains fallback logic to fetch live data from the vault contracts, but this logic is only triggered if vaultConfigs[vault].minDeposit is 0. Since minDeposit is required to be non-zero upon vault creation, this fallback path is effectively unreachable for any existing vault.

```

1  function getTokenConfig(address vault) external view returns (TokenConfig memory) {
2      require(vaults[vault].vault != address(0), "Vault does not exist");
3      // @audit Return stored config if it exists.
4      // @audit-issue This will always execute for an existing vault, returning potentially stale data.
5      if (vaultConfigs[vault].minDeposit > 0) {
6          return vaultConfigs[vault];
7      }
8      // @audit This fallback logic is unreachable for existing vaults.
9      // ...
10 }
```

Recommendation(s): Consider removing the caching layer in the getTokenConfig function. Instead, the function should always fetch the current configuration values directly from the relevant source contracts.

Status: Fixed

Update from the client: This finding is no longer applicable as the VaultFactory.sol contract has been deprecated and removed from our protocol architecture. All vault configurations are now managed directly. Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.21 [Medium] Tracking custodians by custodianId could be inaccurate

File(s): LSTokenVaultStorage.sol

Description: The LSTokenVaultStorage contract manages a list of custodians using a dynamic array named custodians. The _addCustodian(...) function assigns a custodianId to each new custodian based on the current length of the array. The _removeCustodian(...) function uses a "swap with last and pop" method to efficiently remove a custodian from the array.

The issue arises from how custodianId is assigned and managed. The _addCustodian(...) function assigns custodianId as custodians.length, which is simply the next available index in the array. However, the _removeCustodian(...) function's "swap and pop" logic does not maintain a stable mapping of custodianId to a specific custodian. When an element is removed from the middle of the custodians' array, the element at the last position is moved to fill the vacated slot, and the array is then shortened. This means that the custodianId (which is the index in the array) for the last custodian is now associated with a new wallet, and the custodianId for the custodian that was just moved now refers to a different wallet.

For example, if the custodians array has three elements at indices 0, 1, and 2, and the custodian at index 1 is removed, the custodian from index 2 is moved to index 1. From this point on, the custodianId of 1 refers to a different wallet than it did before the removal. This can lead to a state where the custodianId no longer accurately tracks its intended custodian, which could cause confusion and potential inaccuracies in any off-chain systems that rely on custodianId.

Recommendation(s): Consider implementing logic to generate a unique ID and mapping custodian information based on their unique IDs.

Status: Acknowledged

Update from the client: The custodianId, which corresponds to the custodian's index in the array, is not designed to be a permanent, unique identifier. The single source of truth for the current custodian set is always the on-chain custodians array itself. Any off-chain process is expected to query the contract directly (e.g., using the getAllCustodians view function) to get the current and accurate state of custodian wallets and their corresponding indices. Relying on historical event data to reconstruct the current state is not the intended integration pattern for this system. Therefore, we can consider the current implementation to be correct.

6.22 [Medium] _cleanupOldRequests can skip elements due to incorrect loop logic

File(s): [UnstakeManager.sol](#)

Description: The _cleanupOldRequests(...) function is designed to clean up old, processed unstake requests from the queuedRequestIds array. The function iterates through the array and removes elements that are PROCESSED and the request timestamp has expired.

The issue is that the function uses a standard for loop to iterate while simultaneously removing elements from the array. This could result in two problems.

While the batch parameter limits the iteration of the for loop, if it loops for the whole length of the array, it would revert as elements were removed during earlier iterations and hence the array length would have reduced.

The second problem is due to the logic of removing elements from the array, where it swaps the last element with the current index and pops the last element. As a result, the swapped element will not be checked for the condition to remove.

This could result in some old requests remaining in the queue even after being qualified to be removed.

The underlying issue is removing elements from the array while iterating over it.

Recommendation(s): Consider removing elements outside the main loop.

Status: Fixed

Update from the client: Removed the function entirely. This cleanup mechanism was redundant, as the primary operational logic already ensures requests are immediately and correctly removed from the queue upon being processed.

Commit: [9054a08bb70e88933c206e303766ee7fab8e1a20](#)

6.23 [Medium] depositFor checks for the wrong emergency controller status

File(s): [TokenSilo.sol](#)

Description: The TokenSilo contract is responsible for handling pending withdrawals from the main protocol. When a user requests to withdraw funds, they are transferred to the TokenSilo contract, where they remain during a cooling-off period before the user can claim them. The depositFor function is used to transfer funds from the vault to the TokenSilo.

The emergencyController is a contract that can be used to pause certain protocol functionalities in case of an emergency. The possible states are FULL_PAUSE, DEPOSITS_PAUSED, and WITHDRAWALS_PAUSED.

The depositFor function from the TokenSilo contract is used to transfer funds to this contract for pending withdrawals. Since this function is part of a withdrawal workflow, it should be checked against WITHDRAWALS_PAUSED and FULL_PAUSE states. However, the code checks for DEPOSITS_PAUSED instead, which is not correct.

As a result, while emergencyController puts the protocol on pause for deposits, the withdrawals are also blocked.

Recommendation(s): Consider checking for WITHDRAWALS_PAUSED instead of DEPOSITS_PAUSED.

Status: Fixed

Update from the client: The check in depositFor has been changed to WITHDRAWALS_PAUSED to correctly align with the unstaking workflow and ensure protocol consistency during an emergency pause. Commit: [60fe88a860fb35f172610fb9f50f7a2a5d9b9171](#)

6.24 [Low] The VaultFactory contract can't manage newly created Vault contracts

File(s): VaultFactory.sol

Description: The VaultFactory contract is used to deploy and manage Vault contracts. The owner of the VaultFactory contract is responsible for creating new vaults and managing them. However, when a new Vault is created through the createVault function, the globalAdmin is granted the DEFAULT_ADMIN_ROLE and ADMIN_ROLE roles, not the VaultFactory itself.

```

1  function createVault(...) external onlyRole(ADMIN_ROLE) returns (...) {
2      // ...
3      bytes memory vaultInitData = abi.encodeWithSelector(
4          ILSTokenVault.initialize.selector,
5          underlyingToken,
6          lsTokenAddress,
7          underlyingSymbol,
8          lsTokenSymbol,
9          globalAdmin // @audit-issue The `VaultFactory` is not granted `DEFAULT_ADMIN_ROLE`
10     );
11     vaultAddress = address(new ERC1967Proxy(vaultImplementation, vaultInitData));
12     // ...
13 }
```

This means that the VaultFactory contract is not granted the necessary roles on the new Vault instance. As a result, any function in the VaultFactory that tries to call an admin function on the newly deployed Vault will revert. For example, the addCustodianToVault, updateVaultCustodian, and removeVaultCustodian functions all require the caller to have the ADMIN_ROLE on the Vault contract, which the VaultFactory does not have. This forces the globalAdmin to manually grant the VaultFactory contract the DEFAULT_ADMIN_ROLE on each newly created Vault contract, which is a manual process that can be easily forgotten.

Recommendation(s): Considering granting the admin role for VaultFactory in the _setupRoles function.

Status: Fixed

Update from the client: This finding is no longer applicable as the VaultFactory.sol contract has been deprecated and removed from our protocol architecture. Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.25 [Low] Configuration changes can occur on vault during emergency

File(s): VaultManager.sol

Description: The VaultManager contract is a stateless administrative module designed to configure and manage the associated LSTokenVault. It provides functions that allow an ADMIN_ROLE to change parameters, such as deposit limits, fees, and staking/unstaking statuses. The contract is designed to work in conjunction with the EmergencyController, which can globally pause the protocol to prevent malicious activity or mitigate a vulnerability.

The issue is that while the VaultManager contract itself implements Pausable and will be paused by the EmergencyController, the functions that it calls on other contracts, such as LSTokenVault and UnstakeManager, are not necessarily paused. The VaultManager's functions that modify parameters do not check the emergency state. This creates a scenario where an administrator can still modify crucial contract settings even when the system is in an emergency PAUSED state.

Recommendation(s): Fixed

Status: Unresolved

Update from the client: The VaultManager is an administrative contract, and its functions are critical tools for crisis management. During an emergency, such as extreme market volatility or a potential threat, the admin team must retain the ability to execute protective measures. This includes actions like lowering deposit limits, disabling staking entirely, or adjusting fees. Disabling these administrative functions during a pause would severely hinder our ability to respond effectively and safeguard the protocol. The emergency pause is designed to halt user-facing operations, not the administrative controls required to manage the situation.

6.26 [Low] Duplicate custodian entries can be added

File(s): LSTokenVaultStorage.sol

Description: The _addCustodian(...) function is used to add new custodians to the custodians array. The function assigns a new custodian to the custodian and then pushes a new CustodianData struct to the array.

The current implementation of the _addCustodian(...) function does not include a check to see if a wallet address is already present in the custodians array. This means that an administrator could accidentally add the same custodian multiple times. This leads to duplicate entries in the array, which can cause confusion and lead to incorrect total allocation calculations.

Recommendation(s): Consider adding a check at the beginning of the _addCustodian(...) function to ensure that the provided wallet address does not already exist in the custodians collection.

Status: Fixed

Update from the client: The _addCustodian function in LSTokenVaultStorage now includes a check to ensure the provided wallet address does not already exist in the custodians array before adding it. Commit: [35cbe6130426f2be2be74d25dfa40eb46322a546d](#)

6.27 [Low] Fee-on-transfer and rebase tokens

File(s): [LSTokenVault.sol](#), [UnstakeManager.sol](#)

Description: Currently, the DEFx LST application is not compatible with fee-on-transfer and rebase tokens. Such tokens would break the internal calculations.

Recommendation(s): Consider creating vaults only for the tokens with static balances. Alternatively, if such tokens should be available in the vault, the protocol calculations should be adjusted accordingly to handle such tokens correctly.

Status: Acknowledged

Update from the client: Acknowledged. This is a correct assessment and a deliberate design choice. The protocol is intended for use only with standard, non-rebasing ERC20 tokens that do not charge a fee on transfer. We will explicitly state this limitation in our official documentation to ensure users only deposit compatible assets.

6.28 [Low] Inefficient `_removeFromQueue()` function

File(s): [UnstakeManager.sol](#)

Description: The function `_removeFromQueue()` removes the request object from the `queuedRequestIds` array. However, this operation is inefficient, since in the worst case, to remove each ID, the whole array must be searched independently. In extreme examples when the array is large, this may cost a lot of gas, or even lead to out-of-gas condition. Additionally, the management of the array `queuedRequestIds` does not behave like a queue, since elements can be removed in any order.

Recommendation(s): Consider using mapping instead of array for `_removeFromQueue()` mechanism.

Status: Acknowledged

Update from the client: Acknowledged. The current `_removeFromQueue` implementation uses a "swap-and-pop" pattern, which, while requiring an array iteration, is a standard and reasonably gas-efficient method for unordered arrays. We accept this finding as a potential for future gas optimisation, but given the low severity and the unlikelihood of the queue growing to a size where this would cause an out-of-gas issue, we will not be implementing a refactor at this time.

6.29 [Low] Potential to bypass early withdrawal fee

File(s): [TokenSilo.sol](#)

Description: The `earlyWithdraw` function allows a user to withdraw their tokens from the silo before the unlock date. When a user performs an early withdrawal, a fee is calculated based on the withdrawal amount and the configured unlock fee.

The fee calculation uses integer division, which can lead to a precision loss. The fee amount is calculated as $(\text{amount} * \text{unlockFee}) / 10000$. If the product of the withdrawal amount and the unlock fee is less than 10000, the result of the division will be zero. This means that a user can avoid paying the early withdrawal fee by making a series of small withdrawals.

Users can exploit this issue by making multiple small withdrawals instead of a single large one, thus avoiding the fees.

The financial impact is very low since gas costs for multiple transactions would likely outweigh any savings from avoiding fees on small amounts. However, this is still a valid issue as it doesn't correctly implement the intended fee mechanism.

Recommendation(s): Consider processing early withdrawal for the whole amount in the withdrawal request.

Status: Fixed

Update from the client: Acknowledged. The new `earlyWithdraw()` function in the `UnstakeManager` contract compels the user to withdraw their entire balance from the `TokenSilo` in a single transaction. This architecture prevents a user from splitting their withdrawal into multiple smaller transactions to bypass the fee. Therefore, no code change is required as the recommended behaviour is already implemented. Commit: [ee6ae008f1ca4be8478654a65440168ac549a7ea](#)

6.30 [Low] The `maxUserDeposit` limit can be easily avoided

File(s): [LSTokenVault.sol](#)

Description: The `maxUserDeposit` is compared against the user's current LST balance. However, this check can be easily avoided, since users can transfer LST to another address before deposit.

Recommendation(s): Consider tracking the user's LST deposit through a global mapping.

Status: Acknowledged

Update from the client: Acknowledged. We accept this as a design trade-off. The `maxUserDeposit` limit is intended as a soft deterrent against the concentration of capital in a single address, not a strict, Sybil-resistant security measure. Any purely on-chain solution, including the suggested mapping, can be circumvented by using multiple wallet addresses.

6.31 [Low] The function `_cleanupOldRequests()` doesn't work correctly

File(s): `UnstakeManager.sol`

Description: The function `_cleanupOldRequests()` loops over the elements of `queuedRequestIds` array to remove requests with status `PROCESSED` that are older than a month. However, when the request is given `PROCESSED` status, it is always removed from `queuedRequestIds`. Therefore, the `_cleanupOldRequests()` never finds out requests that meet conditions.

Recommendation(s): Consider refactoring or removing the `_cleanupOldRequests()` function.

Status: Fixed

Update from the client: Acknowledged. The `_cleanupOldRequests()` function does not exist in the current implementation, as processed requests are already removed from the queue immediately. Therefore, this issue is not applicable.

Commit: [9054a08bb70e88933c206e303766ee7fab8e1a20](#)

6.32 [Low] `VaultManager` is not granted the required roles on the contracts it manages

File(s): `VaultManager.sol`

Description: The `VaultManager` contract serves as a centralized administrative module. Users with the `ADMIN_ROLE` on this contract can call configuration functions, which in turn delegate the calls to the appropriate underlying contracts such as `LSTokenVault`, `UnstakeManager`, and `TokenSilo`. For example, an admin can call the `setMaxTotalDeposit` function on `VaultManager` to configure the `LSTokenVault`.

```

1  function setMaxTotalDeposit(uint256 _maxTotalDeposit) external override onlyRole(ADMIN_ROLE) {
2      require(vault != address(0), "VaultManager: vault not set");
3      // @audit-issue The LSTokenVault contract will check if msg.sender (this contract)
4      // @audit-issue has the admin role.
5      ILSTokenVault(vault).setMaxTotalDeposit(_maxTotalDeposit);
6  }
```

However, the functions on the underlying contracts are access-controlled, requiring the caller to possess an administrative role. When `VaultManager` makes a delegated call, the `msg.sender` within the target contract is the address of the `VaultManager` itself, not the original admin's address. The protocol's setup process does not grant the necessary administrative roles to the `VaultManager` contract on the contracts it is supposed to manage.

This means that any administrative call made through the `VaultManager` will revert due to a failed access control check in the target contract. This renders the primary functionality of the `VaultManager` contract unusable, as it cannot perform its intended administrative tasks.

Example: In the `vaultManager`, `setFloatPercent` is invoked by the `ADMIN_ROLE`.

```

1  function setFloatPercent(uint256 _floatPercent) external onlyRole(ADMIN_ROLE) {
2      require(vault != address(0), "VaultManager: vault not set");
3      ILSTokenVault(vault).setFloatPercent(_floatPercent);
4  }
```

But when it executes the call on the vault, the caller is expected to have `MANAGER_ROLE`.

```

1  function setFloatPercent(uint256 _floatPercent) external onlyRole(MANAGER_ROLE) {
2      //...
3  }
```

Recommendation(s): Consider updating the deployment and initialisation procedures to ensure that the `VaultManager` contract is granted the necessary administrative roles on all the contracts it manages, such as `LSTokenVault`, `UnstakeManager`, and `TokenSilo`.

Status: Acknowledged

Update from the client: Acknowledged. Our deployment script (`deploy.js`) already addresses this by explicitly granting the necessary `MANAGER_ROLE` and `ADMIN_ROLE` to the `VaultManager` on the `LSTokenVault`, `UnstakeManager`, and `TokenSilo` contracts during the post-deployment setup phase. This ensures the `VaultManager` is fully operational as intended.

6.33 [Low] claimsPaused flag not set on revert

File(s): TokenSilo.sol

Description: The withdrawTo(...) function allows users to withdraw their tokens from the TokenSilo contract. The function first checks if there is sufficient liquidity to fulfil the withdrawal. If the availableBalance is less than the requested amount, it sets the claimsPaused flag to true before reverting the transaction with an "insufficient liquidity" error.

```

1  function withdrawTo(address recipient, uint256 amount) external {
2      require(msg.sender == owner, "Only owner can withdraw");
3      uint256 availableBalance = getTokenBalance() - totalDeposits;
4      // ...
5      if (availableBalance < amount) {
6  ==>         claimsPaused = true;
7              revert("Insufficient liquidity");
8      }
9
10     // ...
11 }

```

The issue is that when a transaction reverts, all state changes made within that transaction are rolled back. This means that even though the code attempts to set claimsPaused to true, this change will not be persisted on the blockchain because the transaction fails. As a result, the claimsPaused flag remains false and the protocol's pause mechanism is not triggered as intended. This could lead to a situation where the contract's liquidity is constantly insufficient, but users are not prevented from making further claims or withdrawals, which could revert due to lack of liquidity.

Recommendation(s): Remove the line setting the claimsPaused to true before reverting.

Status: Fixed

Update from the client: removed the ineffective line of code from both the withdrawTo and earlyWithdrawFor functions. Commit: [ce9e57f36f77a992750f144b81183dc132a5122a](#)

6.34 [Info] Prices are expressed in different decimals between vaults

File(s): LSTokenVault.sol

Description: The price (index) is expressed in decimals of the underlying token. Therefore, the values will differ between the vaults.

Recommendation(s): Consider clearly documenting those differences to avoid user confusion and mistakes.

Status: Mitigated

Update from the client: Acknowledged. This is the intended behaviour of the protocol, as the index is designed to represent the value of the LSToken relative to its specific underlying asset. We will update our official documentation to clearly explain that the index value is relative and must be interpreted in the context of the specific underlying token's decimals to avoid any confusion for users or integrators.

6.35 [Info] Storage for LSTokenVault

File(s): LSTokenVault.sol, LSTokenVaultStorage.sol

Description: In the current implementation, the storage layout for the vault is shared across LSTokenVault and LSTokenVaultStorage contracts. As the gap is added into the LSTokenVaultStorage, which is inherited into LSTokenVault, leaving slots for new variables to be added to the Vault contract.

A better manageable implementation would be to define the storage layout in the LSTokenVaultStorage contract and isolate the logic implementation in the LSTokenVault contract.

Recommendation(s): Define a namespace for storage. This would be the recommended approach for Upgradeable contracts. This minimises the possibility of storage collisions.

Status: Acknowledged

Update from the client: The current pattern, where LSTokenVault inherits its state from LSTokenVaultStorage and utilises a storage gap, is a deliberate design choice. This approach was chosen for its simplicity and clarity, avoiding the added architectural complexity of a separate namespaced storage contract. While we acknowledge that the namespaced storage pattern offers enhanced protection against storage collisions in highly complex systems, we will be retaining the existing storage pattern to avoid major code changes at this time.

6.36 [Info] Unused _maxTransactionPercentage

File(s): LSTokenVault.sol

Description: The _maxTransactionPercentage is set in the function setFlashLoanProtection() but never used.

Recommendation(s): Consider using or removing unused variable setFlashLoanProtection.

Status: Fixed

Update from the client: The unused maxTransactionPercentage variable and its related logic have been removed from the contracts to eliminate dead code. Commit: [3a4efdb7a86e7dc5c4b724095513bd10adf0cb4d](#)

6.37 [Info] Vault status in the VaultFactory is ineffective

File(s): VaultFactory.sol

Description: The VaultFactory contract is responsible for deploying Vault contracts. For each vault created, the factory maintains a status to track whether the vault is active or inactive. This functionality suggests an administrative capability to enable or disable vaults.

However, this status is confined to the VaultFactory contract and is not propagated to or checked by the individual Vault contracts. Consequently, a vault that is marked as "inactive" in the factory will continue to function without any restrictions, allowing users to deposit and withdraw funds as usual.

This leads to a misleading state within the VaultFactory, as the tracked status has no actual effect on the operational capabilities of the Vault contracts. The feature, as it stands, is dead code and provides a false sense of control over the deployed vaults.

Recommendation(s): Consider either enforcing the vault status or removing it from VaultFactory.

Status: Fixed

Update from the client: Acknowledged. This finding is no longer applicable. The VaultFactory.sol contract and its status-tracking feature have been deprecated and removed from our protocol architecture. Vault activation and deactivation are now managed directly through the setStakeEnabled and setUnstakeEnabled functions on the LSTokenVault itself, controlled by the VaultManager. Commit: [ab8d2debbd881f207c22619a8a26390558c0f4a7](#)

6.38 [Info] Vaults can be left without custodians

File(s): VaultFactory.sol

Description: The VaultFactory contract is responsible for creating and managing ILSTokenVault instances. During vault creation, the createVault function enforces a critical invariant. Every vault must be initialised with at least one custodian. This ensures that there are always designated parties to manage the vault's operations.

The problem lies within the removeVaultCustodian function, which is intended to be used by an address with the ADMIN_ROLE to remove a custodian from a specified vault. This function calls removeCustodian on the vault but fails to verify the number of custodians remaining after the operation.

```

1 function removeVaultCustodian(address vault, uint256 custodianId) external
2   onlyRole(ADMIN_ROLE)
3 {
4   require(vaults[vault].active, "Vault not active");
5   // @audit-issue It does not check if at least one custodian remains after removal.
6   ILSTokenVault vaultContract = ILSTokenVault(vault);
7   vaultContract.removeCustodian(custodianId);
8 }

```

This omission creates an inconsistency with the vault creation logic, which explicitly requires at least one custodian.

```

1 / @audit Vault creation requires at least one custodian.
2 require(config.custodians.length > 0, "Must have at least one custodian");

```

Recommendation(s): Consider adding a check within the removeVaultCustodian function to ensure that the removal of a custodian does not leave the vault with zero custodians.

Status: Acknowledged

Update from the client: Acknowledged. This finding is no longer applicable as the VaultFactory.sol contract has been deprecated and removed from our protocol architecture. Custodian management is now handled directly through the LSTokenVault contract, which contains a check preventing the removal of the last custodian.

6.39 [Info] setFlashLoanProtection is missing in TokenSilo

File(s): [TokenSilo.sol](#), [ITokenSilo.sol](#)

Description: setFlashLoanProtection function is declared in the ITokenSilo interface, but was not implemented in the TokenSilo contract.

Recommendation(s): Consider removing the function declaration in the interface.

Status: Fixed

Update from the client: The setFlashLoanProtection function is not relevant to the TokenSilo contract and was mistakenly included in its interface. Removed the function declaration from ITokenSilo.sol to correct the mismatch. Commit: [2f1cf06](#)

6.40 [Info] transferCollateral function calls a non-existent function

File(s): [VaultManager.sol](#)

Description: The VaultManager contract contains an administrative function transferCollateral, which is intended to be used by an address with the ADMIN_ROLE to transfer underlying assets from the LSTokenVault contract.

The problem is that transferCollateral attempts to call the transferUnderlying(to, amount) function on the vault address. However, the LSTokenVault contract, which is the implementation for the vault, does not contain the transferUnderlying function. Although this function is defined within the ILSTokenVault interface, it is not implemented in the concrete contract.

```
1 function transferCollateral(address to, uint256 amount) external override onlyRole(ADMIN_ROLE) {  
2     require(vault != address(0), "VaultManager: vault not set");  
3     // @audit-issue This call will always revert because the `transferUnderlying(...)`  
4     // @audit-issue function is not implemented in the `LSTokenVault.sol` contract.  
5     ILSTokenVault(vault).transferUnderlying(to, amount);  
6     emit AdminTransfer(to, amount);  
7 }
```

This discrepancy makes the transferCollateral function unusable. Any call to this function will revert because the target function does not exist on the implementation contract. This indicates a mismatch between the protocol's interface and its implementation, rendering a key administrative feature non-functional.

Recommendation(s): Consider aligning the contract's interface with its implementation contract.

Status: Fixed

Update from the client: Acknowledged and fixed. We have implemented the missing transferUnderlying function in the LSTokenVault.sol contract, protected by the MANAGER_ROLE. This aligns the implementation with the interface and makes the transferCollateral function in VaultManager fully operational as intended. Commit: [92f38d9cba48ee9df75e36eb9dc8bdf56618f2cb](#)

6.41 [Info] conflicting state control between ADMIN and EMERGENCY Roles

File(s): EmergencyControl.sol

Description: The EmergencyController contract is designed to manage the protocol's state during emergencies. It defines a highly restrictive recoveryModeActive state that is triggered after a 24-hour waiting period, initiated by the triggerCircuitBreaker(...) function, which can only be called by the EMERGENCY_ROLE. This function also immediately places the system into a FULL_PAUSE state. However, the resumeOperations() function, which can be called by the ADMIN_ROLE, is capable of reverting the system to the NORMAL state.

This creates a conflicting scenario. When the EMERGENCY_ROLE triggers the circuit breaker, setting the state to FULL_PAUSE and starting the 24-hour countdown for recovery mode, the ADMIN_ROLE can immediately call resumeOperations() to override the state and restore the system to NORMAL. This action resets the pause state but does not reset the recoveryModeActivationTime. As a result, the ADMIN can repeatedly revert the state to NORMAL, while the timer for recovery mode activation continues to count down in the background. If the ADMIN does not intervene for a full 24 hours, the recovery mode will activate, potentially catching the ADMIN off guard and creating an unexpected state for the protocol. This conflict in permissions between two different roles can lead to confusion and unintended behaviour in an emergency situation.

```
1 function resumeOperations() external override onlyRole(ADMIN_ROLE) {
2     // ...
3     // @audit-issue This function resets the emergency state but not the recovery timer.
4     _emergencyState = EmergencyState.NORMAL;
5     // ...
6 }
7
8 function triggerCircuitBreaker(string calldata reason) external override onlyRole(EMERGENCY_ROLE) {
9     _emergencyState = EmergencyState.FULL_PAUSE;
10    // ...
11    // @audit The timer for recovery mode starts here.
12    recoveryModeActivationTime = block.timestamp;
13    // ...
14 }
```

Recommendation(s): Consider adding a check in resumeOperations for recoveryModeActivationTime to be 0 before the state can be set to Normal.

Status: Fixed

Update from the client: The ADMIN_ROLE is no longer able to override a scheduled recovery mode simply by resuming operations. To exit a circuit-breaker event, the ADMIN must now follow a deliberate, two-step process:

First, they must explicitly call deactivateRecoveryMode(), which has been updated to allow for the cancellation of a scheduled (not just active) recovery. This action resets the recovery timer.

Only after the timer has been reset can the ADMIN call resumeOperations() to bring the protocol back to a normal state. Commit: [0265e3afc32c524713bdba8ef472348a6ef3102d](#)

6.42 [Info] versioning and upgrading are not synchronized

File(s): LSToken.sol, *

Description: The protocol contract is designed to be upgradeable using the UUPS proxy pattern. They implement separate mechanisms for managing contract versions and upgrading.

```

1  function updateVersion(string memory _newVersion) external onlyRole(ADMIN_ROLE) {
2      version = _newVersion;
3      emit VersionUpdated(_newVersion);
4  }
5
6  function _authorizeUpgrade(address newImplementation) internal override onlyRole(ADMIN_ROLE) {
7      require(newImplementation != address(0), "LSToken: invalid implementation");
8
9      require(upgradeControl.requested, "LSToken: upgrade not requested");
10     require(block.timestamp >= upgradeControl.requestTime + UPGRADE_TIMELOCK, "LSToken: timelock not expired");
11
12     // Reset the upgrade flag after a successful authorisation.
13     upgradeControl.requested = false;
14
15     emit UpgradeAuthorized(newImplementation, version);
16 }
  
```

The issue is that these two functionalities are completely independent. An administrator could update the version string without performing an actual contract upgrade. Conversely, an upgrade could be performed without the version string being updated. This creates a risk of inconsistent state and confusion for external systems. When a version is updated, it typically signifies that the contract's logic has changed.

Recommendation(s): Consider managing the version in `_authorizeUpgrade` as part of the upgrade.

Status: Fixed

Update from the client: Implemented a fully on-chain solution by replacing the manual version string with an automatic uint256 version counter. This counter increments with every successful contract upgrade, ensuring the on-chain version is always a reliable and accurate reflection of the deployed code. Commit: [7fa826de503ad71388c4ceeb80c2809a4a57c776](#)

6.43 [Best Practice] CEI pattern

File(s): *

Description: Some functions do not follow the CEI pattern. An example of this is the function `UnstakeManager.requestUnstake()`, which burns LST token at the top of the function, but since it is an external interaction, it should be done last.

Recommendation(s): Consider keeping the CEI pattern whenever possible.

Status: Acknowledged

Update from the client: Acknowledged. We agree that adhering to the CEI pattern is a crucial security best practice. However, in the case of `UnstakeManager.requestUnstake()`, the external call is to `IsToken.burnFrom()`, which is a trusted contract deployed and controlled by our own protocol. The LSToken implementation is a standard ERC20 token without any callbacks that would introduce a reentrancy risk. While the pattern is not strictly followed, we have assessed the specific context and determined that there is no reentrancy vulnerability. We accept this as a low-risk design choice.

6.44 [Best Practice] Percentages calculations

File(s): *

Description: Currently, percentages are calculated using a denominator 100, while in other places, it's basis points where the denominator is 10000.

Recommendation(s): Consider keeping calculations of percentages consistent across the contracts.

Status: Acknowledged

Update from the client: Acknowledged. This is a deliberate design choice. We use a percentage (denominator 100) for broader protocol-level fees like yield distribution in the LSTokenVault. For user-facing financial parameters that require finer granularity, such as the early withdrawal penalty in the TokenSilo, we use basis points (denominator 10,000) as is standard practice. We believe this approach is appropriate for each specific use case.

6.45 [Best Practice] Using gaps

File(s): *

Description: The upgradeable contracts that are inherited contain the `__gap` mapping to allocate additional space for future variables added during upgrade. The recommended size of the gap should add up with the existing storage slot to sum up to a round number, e.g. if the contract populates 2 slots with variables, the gap should be `__gap[48]` so that the sum of taken space is 50. This helps manage gap size during upgrades, since all contracts should have taken, e.g. 50 slots. More information and examples can be found in the [OpenZeppelin article](#).

Recommendation(s): Consider using gaps in the way described above.

Status: Fixed

Update from the client: We have adopted the best practice recommendation to standardise our contract storage layouts. The `__gap` variable in each upgradeable contract has been recalculated to ensure the total number of storage slots is a consistent and predictable number (60). Commit: [524179de53b9ab509c3ddc6f6f3d0b3cf125bc06](#)

6.46 [Best Practices] Lack of Segregation of Duties for Critical Roles

File(s): *

Description: The EmergencyControl contract is a critical component of the protocol, as it likely manages emergency functions or administrative tasks that could have a significant impact on the system. The contract's functions are protected by roles defined using OpenZeppelin's AccessControl library.

The current implementation assigns all roles (PAUSE_ROLE, UNPAUSE_ROLE, HALT_ROLE, UNHALT_ROLE, and WITHDRAW_ROLE) to a single account.

```
1 // @audit All critical roles are assigned to a single account.
2 _setupRole(PAUSE_ROLE, singleAccount);
3 _setupRole(UNPAUSE_ROLE, singleAccount);
4 _setupRole(HALT_ROLE, singleAccount);
5 _setupRole(UNHALT_ROLE, singleAccount);
6 _setupRole(WITHDRAW_ROLE, singleAccount);
```

While this centralises control, it violates the principle of segregation of duties, which is a key security and best practices concept. By giving a single account full control over all emergency functions, the protocol introduces a single point of failure. If the private key for this single account is compromised, a malicious actor could gain complete control over all critical functionalities, including pausing/unpausing and halting/unhalting the system, and withdrawing funds. This increases the risk of insider threats, human error, and external attacks.

Recommendation(s): Consider implementing a robust system for segregating roles and responsibilities. Each role should be assigned to a different account or a multi-signature wallet.

Status: Acknowledged

Update from the client: Upon deployment, a single, highly-secured multi-signature wallet will be assigned the ADMIN_ROLE. This admin will then be responsible for delegating the MANAGER_ROLE, EMERGENCY_ROLE, and other operational roles to distinct, separate entities as a required post-deployment setup step. While this centralises authority at the moment of initialisation, it provides the necessary operational flexibility for our team to manage role holders over the lifecycle of the protocol.

6.47 [Best Practices] Missing Function to Update version

File(s): [LSTokenVault.sol](#), [VaultFactory.sol](#)

Description: The LSTokenVault and VaultFactory contracts are implemented with the UUPS upgradeable pattern. Both contracts have a version state variable to track their current implementation version. This variable is intended to provide transparency and an easy way for off-chain systems to identify the currently deployed version.

However, a function to update this version state variable is missing from both contracts. This means that while the contract's logic can be upgraded, the version variable will remain static, leading to a mismatch between the contract's internal version number and its actual implementation. This can cause confusion and potential issues for any external services that rely on this variable for accurate version tracking. It is a best practice to keep all state variables that represent the contract's state consistent with its logic.

Recommendation(s): Consider adding the logic to increment the version in the upgrade function.

Status: Fixed

Update from the client: To ensure the contract's version is always synchronised with its implementation, we have replaced the static version string with an automated uint256 version counter in both LSTokenVault and VaultFactory. This counter is now automatically incremented within the `_authorizeUpgrade` function. Commit: [7fa826de503ad71388c4ceeb80c2809a4a57c776](#)

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Defx's documentation

The **Defx's LST Contracts** documentation is primarily provided through the `readme.md` in the repository. The codebase itself has comments facilitating for code comprehension and auditability. Additionally, the Defx team was responsive and available for internal sync calls, promptly addressing all questions and discussion points raised by the Nethermind Security team.

8 Test Suite Evaluation

8.1 Tests Output

```
> npx hardhat test
6-Decimal Scaling Integration Test
  should correctly scale a 6-decimal deposit to an 18-decimal LSToken
  should correctly scale an 18-decimal unstake to a 6-decimal silo balance
  should correctly scale 6-decimal yield and 18-decimal fees
  should correctly scale 6-decimal slippage checks
  should correctly scale 6-decimal early withdrawal fees (Preview)
  should correctly scale 6-decimal early withdrawal fees (Execute)
  should return consistent native decimal values from all view functions

Contract Deployment Validation (Decimals > 18)
  should revert when deploying LSTokenVault with a 19-decimal token
  should revert when deploying TokenSilo with a 19-decimal token
  should revert when deploying UnstakeManager with a 19-decimal token

Emergency Controller Integration Test
  DEPOSITS_PAUSED State
    should revert vault.deposit()
    should REVERT vault.addYield()
    should NOT revert silo.depositFor()
  WITHDRAWALS_PAUSED State
    should revert vault.requestUnstake()
    should revert unstakeManager.claim()
    should revert unstakeManager.earlyWithdraw()
    should revert silo.withdrawTo()
    should revert silo.earlyWithdrawFor()
    should NOT revert vault.deposit()
  FULL_PAUSE State
    should revert vault.deposit()
    should revert vault.requestUnstake()
    should revert unstakeManager.claim()
  RECOVERY_MODE_ACTIVE State
    should revert vault.deposit()
    should revert vault.requestUnstake()
    should revert vault.addYield()
    should revert unstakeManager.claim()
    should revert silo.depositFor()

EmergencyController
  Initialization
    should set the initial admin correctly and grant all roles
    should initialize in the 'NORMAL' state
  System State Control
    pauseDeposits: should allow EMERGENCY_ROLE to pause deposits
    pauseWithdrawals: should allow EMERGENCY_ROLE to pause withdrawals
    pauseAll: should allow EMERGENCY_ROLE to pause all operations
    resumeOperations: should allow ADMIN_ROLE to resume normal operations
    should prevent non-authorized roles from changing the state
  Circuit Breaker and Recovery Mode
    triggerCircuitBreaker: should immediately pause all and schedule recovery mode
    activateRecoveryMode: should fail if called before the timelock expires
    activateRecoveryMode: should succeed after the 24-hour timelock
    deactivateRecoveryMode: should allow ADMIN_ROLE to deactivate recovery mode
    resumeOperations: should fail if recovery mode is active

Protocol Integration Tests
  Happy Path: Full Staking and Unstaking Flow
    should allow a user to deposit, earn yield, unstake, and claim successfully
  Early Withdrawal Flow
    should allow a user to unstake and withdraw early for a fee
  Emergency Scenario
    should block deposits and withdrawals when Circuit Breaker is triggered
  Complex State Interaction
    should correctly value an unstake request made before a large yield event

LSToken
  Initialization
    should deploy successfully and set the correct name and symbol
```

```

    should grant DEFAULT_ADMIN_ROLE and ADMIN_ROLE to the deployer
    should set the initial contract version to 1
    should initialize with zero total supply
ERC20 Functionality
    should correctly return 18 decimals
Transfers
    should transfer tokens between accounts
    should fail to transfer more tokens than the sender balance
    should fail to transfer tokens to the zero address
Allowances and transferFrom
    should correctly approve an allowance
    should allow a spender to transfer tokens on behalf of the owner
    should fail if the transferFrom amount exceeds the allowance
Access Control: Minting and Burning
Minting
    should allow an account with MINTER_ROLE to mint tokens
    should fail if an account without MINTER_ROLE tries to mint
    should fail to mint to the zero address
Burning
    should allow an account with MINTER_ROLE to burn tokens from another account with allowance
    should fail if an account without MINTER_ROLE tries to burn
    should fail to burn without a sufficient allowance
Upgradeability
    should allow an admin to request an upgrade
    should not allow a non-admin to request an upgrade
    should allow an admin to cancel an upgrade
    should fail to upgrade before the timelock expires
    should successfully upgrade after the timelock period

LSTokenVault
Initialization
    should set the correct initial values
    should grant all roles to the initial admin
deposit
    should allow a user to deposit and mint lsTokens at a 1:1 ratio initially
    should fail if staking is disabled
    should fail if the deposit amount is below the minimum
    should fail if the global deposit limit is reached
    should enforce the user deposit limit
Configurable Vesting Duration
    should respect the updated yieldVestingDuration for withdrawal locks
addYield
    should correctly calculate fees and increase the targetIndex
    should fail if yield is vesting
    should vest the index linearly over time
requestUnstake
    should fail if the withdrawal lock is active
    should successfully request an unstake after the lock expires
Forfeited Yield (Early Unstake)
    should redirect forfeited yield from an early unstake directly to protocol fees
Custodian Management
    should allow an admin to add, update, and remove custodians
    should prevent total allocation from exceeding 100%
Emergency Pause
    should prevent deposits when paused by admin
    should prevent deposits when deposits are paused by the EmergencyController
Input Validation and Edge Cases
    should revert when trying to add zero yield
    should revert if yield is too low to register an index change
    should revert when trying to deposit zero
    should revert when trying to unstake zero
Deposit distribution to custodians
    sends 80% to a single custodian and keeps 20% as float
    splits across multiple custodians and leaves float; handles rounding on small deposits
    reverts deposits when Recovery Mode is active (EC pauses deposits)
    baseline: with no custodians, entire deposit remains in vault (float)
    reverts when adding a custodian with zero wallet
    may emit no transfer for extremely tiny deposits due to integer rounding
    sends funds to the exact custodian wallet (not feeReceiver/admin unless set as custodian)
    works with a 6-decimals underlying (USDC-style) for 10 units -> 8 to custodian

TokenSilo
Initialization
    should set the correct addresses and default config
Fund Management

```

```

    depositFor: should allow the VAULT_ROLE to deposit funds for a user
    withdrawTo: should allow the VAULT_ROLE to withdraw funds for a user
    should fail to withdraw more than the user's balance
Early Withdrawal
    should allow early withdrawal and correctly deduct fees
    should fail if early withdrawal is not enabled
Security and Liquidity
    should automatically pause claims if liquidity drops below the threshold
    should enforce the daily withdrawal rate limit
    should automatically un-pause claims if liquidity is restored
Admin Functions
    should allow an admin to change the unlock fee
    should not allow a non-admin to change the unlock fee
    should allow an admin to rescue accidentally sent tokens
Input Validation
    should revert on depositFor(user, 0)
    should revert on withdrawTo(user, 0)
    should revert on earlyWithdrawFor(user, 0)

UnstakeManager
  Initialization
    should set the correct addresses and grant VAULT_ROLE to the vault
    should set default configuration values
  Unstake Request Lifecycle
    Step 1: requestUnstake - should create a request and burn lsTokens
    Step 2: markRequestsForProcessing - should change request status to PROCESSING
    Step 3 & 4: processUserUnstake & claim - full flow for a single user
    should fail to claim before the cooldown period ends
  Admin and Manager Functions
    should allow a manager to cancel an unstake request
    should prevent a manager from canceling an unstake while yield is vesting
    should allow an admin to set the cooldown period
    should fail if a non-admin tries to set the cooldown period
  Batch Processing
    should process requests in batches correctly

VaultManager
  Initialization and Setup
    should initialize with the correct vault and admin addresses
    should allow the admin to set the UnstakeManager, TokenSilo, and EmergencyController addresses
    should emit events when setting contract links
  Proxied LSTokenVault Functions (ADMIN_ROLE via VaultManager)
    setMaxTotalDeposit: should correctly call on the vault
    setFeePercent: should correctly call on the vault
    setStakeEnabled: should correctly call on the vault
    setYieldVestingDuration: should correctly call on the vault
    setVaultRateLimits: should correctly call on the vault
    setFlashLoanProtection: should correctly call on the vault
    setFloatPercent: should correctly call on the vault
    should prevent non-admins from calling vault configuration functions
  Proxied UnstakeManager Functions (ADMIN_ROLE via VaultManager)
    setCooldownPeriod: should correctly call on the UnstakeManager
    setMinUnstakeAmount: should correctly call on the UnstakeManager
  Proxied TokenSilo Functions (ADMIN_ROLE via VaultManager)
    setSiloRateLimit: should correctly call on the TokenSilo
  Manager-Specific Functions (MANAGER_ROLE via VaultManager)
    withdrawFees: should allow a manager to call on the vault
    recordCustodianFundsReturn: should allow a manager to call on the vault
    should prevent a non-manager from calling withdrawFees
    should prevent a non-manager from calling recordCustodianFundsReturn
    managerEarlyWithdraw: should allow a manager to call on UnstakeManager
    should prevent non-managers from calling managerEarlyWithdraw
  Admin-Specific Functions (ADMIN_ROLE via VaultManager)
    transferCollateral: should allow an admin to call on the vault
    should prevent non-admins from calling transferCollateral
143 passing (8s)

```

8.2 Automated Tools

8.2.1 AuditAgent

All the relevant issues raised by the AuditAgent have been incorporated into this report. The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at

<https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.